

University of Reading
Department of Computer Science

MARS: A Multi-model Agent for Remote Sensing

Weijie Cui

Supervisor: Dr. Muhammad Shahzad

A report submitted in partial fulfilment of the requirements of
the University of Reading for the degree of
Master of Science in *Data Science and Advanced Computing*

September 22, 2025

Declaration

I, Weijie Cui, of the Department of Computer Science, University of Reading, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of UoR and public with interest in teaching, learning and research.

Weijie Cui
September 22, 2025

Abstract

The exponential growth of **Remote Sensing (RS)** technologies has transformed Earth observation, enabling high-resolution imaging for applications like environmental monitoring, urban planning, and disaster rescue. However, interpreting these images remains challenging due to their complexity, multi-scale objects, and dense spatial-spectral information. Traditional methods rely on single-pass visual Artificial Intelligence (AI) models, which struggle with partially obscured, small and shadowed objects. Machine learning (ML) and deep learning (DL) have advanced automated analysis, but limitations persist supervised models require extensive labelled data, while conventional vision-language models (VLMs) lack domain-specific expertise and iterative refinement capabilities. Inspired by the human visual system, to rapidly search and focus on important areas to extract fine details and validate information by prior knowledge, Multi-model Agent for Remote Sensing (MARS) offers a promising solution to interpret images in detail. An Artificial Intelligence Agent (AI Agent) orchestrates a vision model (VM) and a Reinforcement Learning (RL) model, forming the foundation for next-generation remote sensing systems. The RL model generates observation strategies for VM to identify the class and location of objects in images by focusing on different areas multiple times and eventually achieve the visual goal. Experiments demonstrate the robustness of the MARS, which is size insensitive and could detect more partially obscured, small or shadowed targets. In addition, it does not significantly increase GPU computing resources. Code and video demos are available at <https://github.com/WeijieCui/MARS>.

Keywords: Reinforcement Learning, Remote Sensing, AI Agent, MARS

Acknowledgements

I would like to express my deepest appreciation to my supervisor, Dr. Muhammad Shahzad, for his invaluable guidance and constructive criticism. On a personal note, I owe my deepest gratitude to my wife Chuyu Wang and my kids, Yiming and Yidi, for their endless love and encouragement. Finally, I would like to thank all the lecturers, relatives and friends who have helped me with my studies.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Aims and Objectives	2
1.4 Solution Approach	3
1.4.1 Framework Architecture	3
1.4.2 Operational Workflow	4
1.4.3 Methodology for Addressing Aims and Objectives	5
1.4.4 Expected Outcome and Validation	5
1.5 Summary of Contributions and Achievements	5
1.6 Organization of the Report	6
2 Literature Review	8
2.1 Computer Vision Models in Remote Sensing	8
2.2 Agent-based Systems	8
2.3 Remote Sensing Dataset	9
2.4 Critique of Existing Work and the Research Gap	10
2.5 Summary	11
3 Methodology	12
3.1 System Structure	12
3.1.1 Environment: Zoomable Visual Exploration	13
3.1.2 Reinforcement Learning	14
3.1.3 Vision Model	15
3.1.4 User interface	15
3.2 Document Architecture	16
3.2.1 Introduction of The Main Entry	17
3.2.2 Introduction of RL model	20
3.2.3 Introduction of MARS Environment	22
3.3 Datasets	25
3.4 Testing and Validation	27
3.5 Summary	27

4	Results	29
4.1	Metrics for Training RL Model	29
4.2	Benchmark and Metrics for MARS Agent	30
4.3	Comparison of Visualization Results	31
4.4	Summary	35
5	Discussion and Analysis	36
5.1	Experimental Results	36
5.2	Significance of the Findings	36
5.3	Limitations	36
5.4	Practical Values	37
5.5	Summary	37
6	Conclusions and Future Work	38
6.1	Conclusions	38
6.2	Future Work	38
7	Reflection	40
	References	41
	Appendices	42
A	An Appendix Chapter (Optional)	42

List of Figures

2.1	Sample Images for DOTA dataset.	9
2.2	Sample images for DIOR dataset.	10
2.3	Sample image and question pairs for VLBenchmark.	10
3.1	The system structure of our MARS framework.	12
3.2	The Exploration Space of MARS Environment. The window starts from the top layer (zoom level = 1) and gradually moves to the next step based on the prediction strategy. As the zoom level increases, the area of the corresponding window becomes smaller, and the pixels within the window are magnified, and the corresponding details are enhanced, achieving better recognition results. .	14
3.3	The Web Interface of MARS architecture. It includes a result display area on the left, a window and search map in the upper right corner, a control panel in the middle right, and a message display module in the lower right corner. .	16
3.4	Document Architecture of MARS	17
3.5	Main Entry of MARS	18
3.6	Dense urban example, where the main targets are swimming pools and cars. And an example of a dense parking lot, where the main targets are small and large cars.	26
3.7	An example of a sports center including football fields, badminton courts, swimming pools, and small cars. And an example of a body of water, including ships, harbors, and small cars on the ground.	26
3.8	Object type distribution diagram. Mainly including ships, swimming pools, large vehicles, small cars, etc.	27
4.1	Performance of RL Q-table Model-1. Left: Average vehicle found in each episode. Right: Average Reward in each episode.	29
4.2	Performance of RL Q-table Model-2	30
4.3	Sample Results of YOLOv11 and MARS at the Lake	31
4.4	Sample Results of YOLOv11 and MARS at Open Waters	32
4.5	Sample Results of YOLOv11 and MARS at an Airport	33
4.6	Sample Results of YOLOv11 and MARS at an Car Park	34
6.1	Future Work Plan for MARS.	38
A.1	mAP@0.5 Performance of MARS agent and YOLOv11 base model in valuation dataset	42
A.2	mAP@0.5 Performance of MARS agent and YOLOv11 base model in complex valuation dataset	42

List of Tables

4.1	mAP@0.5 Performance of MARS Agent and YOLOv11 in Simple Images . . .	30
4.2	mAP@0.5 of Complex Images	31

List of Abbreviations

3D	Three Dimensions
AI	Artificial Intelligence
CLIP	Contrastive Language-Image Pre-Training
CNN	Convolutional Neural Network
IoT	Internet of Things
IoU	Intersection over Union
mAP	mean Average Precision
MARS	Multi-model Agent for Remote Sensing
RL	Reinforcement Learning
SAM	Segment Anything Model
VM	Vision Model
YOLO	You Only Look Once
ZooVE	Zoomable Visual Exploration

Chapter 1

Introduction

1.1 Background

The exponential growth of **Remote Sensing** (RS) technologies has transformed Earth observation, enabling high-resolution imaging for applications such as environmental monitoring, urban planning, and disaster rescue. However, interpreting these images remains challenging because of their complexity. Remote sensing images can be captured from various altitudes, such as low-altitude drones, high-altitude aircraft, and satellites. They can also be captured using natural light cameras or thermal imaging cameras. Object sizes can also vary greatly, from small objects like people and cars to large aircraft and football fields. The same object captured at different times can also be affected by varying lighting angles, which can affect recognition. Since the advent of LetNet-5 in 1998, computer vision has made long-term progress in object detection, image segmentation, and 3D modeling. The model architecture has also evolved from shallow neural networks to deep neural networks or residual neural networks, and even complex visual language models, such as Contrastive Language-Image Pre-Training (CLIP). But these models rely on single-pass detection, which makes them struggle with partially obscured, small, and shadowed objects. At the same time, as the recognition ability of the model improves, the model needs to contain more parameters and layers, which means more training data and training resources are needed. For instance, the CLIP team trained the largest vision model for 18 days on 592 V100 GPUs and the largest Vision Transformer for 12 days on 256 V100 GPUs. This is considered acceptable, even common, in recent AI model training. But it is not conducive to the practice of AI skills in Internet of Things (IoT) devices, such as a quad-core Arm Cortex A76 processor with 8GB of SDRAM.

1.2 Problem Statement

This essay will investigate the core problem through the following detailed research questions:

Research Question 1: Can an agent AI improve the analytical accuracy of remote sensing image analysis?

- Sub-question 1.1: To what extent can an RL-driven agent enhance the model's ability to detect and classify small, occluded, or rare objects (e.g., small vehicles and ships) compared to a static baseline model?
- Sub-question 1.2: Does the agent-based approach reduce error rates in fine-grained classification tasks (e.g., distinguishing between big vehicles and small vehicles) by focusing computational resources on discriminative regions?

- Hypothesis: The adaptive searching mechanism afforded by the agent will lead to a statistically significant increase in accuracy metrics (mAP, IoU, F1-score) on challenging remote sensing datasets by extracting more relevant visual details.

Research Question 2: Can this enhancement in visual capability be achieved without a proportional increase in computing resource usage?

- Sub-question 2.1: How does the total computational cost (measured in FLOPs, inference time, and memory usages) of the agent-driven model compared to a static model of equivalent accuracy?
- Sub-question 2.2: Does the agent learn to minimize redundant computations? For instance, does it learn to avoid over-processing homogeneous regions like large bodies of water or empty fields?
- Hypothesis: The agent will learn an efficient policy that prioritizes computation, leading to a higher accuracy-per-computation ratio. The system will not achieve superior performance than a static model. While its computation time will increase significantly, it will not require additional resource allocation.

1.3 Aims and Objectives

To address these issues, the project identified some goals:

Aims: The final aim of this project is to investigate, design, and implement a novel agent-driven reinforcement learning framework MARS that dynamically optimizes the analysis of remote sensing images.

- The primary goal is to demonstrate that this adaptive approach can improve the accuracy of fine-grain extraction and distribute the workload compared to conventional static one-pass visual models.

Objectives:

- To design and build the MARS framework integrating an Agent AI with a visual environment, a visual model, and a reinforcement learning model. For the visual environment, it maintains status information such as the state space (e.g., current window, current feature maps, location context), the action space (e.g., left, right, up, down, zoom-in, and zoom-out) and the reward function (balancing task performance gain against computational cost).
- Design and implement a baseline model and the proposed agent-based model for a chosen remote sensing task. This project selected a benchmark task (e.g., small, multiscales and shaded object detection in the DOTA dataset) for evaluation. Then, a state-of-the-art static model (e.g., a YOLO) was chosen as the baseline. Subsequently, we designed and implemented the proposed MARS framework.
- To qualitatively evaluate and compare the performance accuracy of the models. The MARS agent was trained and evaluated on DOTA remote sensing dataset as well as the frozen YOLO version 11 model. It was measured and compared by key accuracy metrics such as mean Average Precision (mAP) for detection tasks or Intersection over the Union area (IoU) and F1-score for classification tasks. This objective directly addresses Research Question 1.

- To quantitatively evaluate and compare the computational efficiency of the models. Profile both models during inference. Measure and compare resources consumed, including inference time (seconds per image), and GPU memory usages. This analysis will test the hypothesis that the agent achieves better performance within an acceptable timeframe and directly addresses Research Question 2.
- To perform a quantitative analysis of MARS's policy and decision-making processes. In order to more intuitively analyze the agent's decision-making process, this project developed a web application to visualize the agent's actions (e.g., where it chooses to look, what regions it selects for high-resolution analysis). This will provide insight into how the agent improves performance, revealing whether it learns intelligent, human-like strategies for analyzing complex scenes.
- To summarize findings, discuss limitations, and propose directions for future research. Finally, this project summarizes the results from Objectives 3, 4 and 5 to form a conclusive arguments, discusses the practical values of the findings, acknowledges the limitations of the current studies and suggests useful steps for further development of this methodology.

1.4 Solution Approach

Inspired by the human visual system, to rapidly search and focus on important areas to extract fine details and validate information by prior knowledge, this project proposes the Multi-model Agent for Remote Sensing (MARS) framework. MARS is an adaptive, Reinforcement Learning (RL)-driven system designed to mimic the human visual system's ability to perform fine-grain analysis. Instead of processing an entire image at a fixed, high resolution, a computationally expensive approach, MARS employs an intelligent agent that learns to strategically search on regions of interest, so that distributing computing resources properly and gaining reasonable accuracy.

The methodology for implementing and evaluating the MARS framework is structured as follows:

1.4.1 Framework Architecture

The MARS framework consists of three core components that interact in a closed-loop system: **The Visual Environment**

- **State Space:** The state represents the agent's current context and view of the input image. It contains an original entire source image, a current image window (or patch) the agent is examining, newfound objects extracted from window by the Visual Model, found objects, which is the set of all previously discovered objects, a scale level of the current window, the coordinate of the center of the window.
- **Action Space:** The agent can take a customized number of actions to change its field of view. The action space is discrete and includes moving left, moving right, moving up, moving down, zoom in, zoom out.
- **Reward Function:** The reward function is essentially for guiding the RL model's learning. It is designed to balance task performance improvement and computational cost. For the Positive Reward, a base reward is given for each new object detection action (+1); a smaller reward is given for classification confidence. If any object was found, a small

reward (+0.1) for a zoom-in action to encourage detailed searching. For the Negative Reward (Cost), a small negative penalty (-0.2) is applied for every action taken without a new object detection.

▪

The Visual Model (VM)

- The VM (YOLO) plays as the “eye” of the agent. Its responsibility is to recognize the objects from the image patch provided by the agent and return the result list, including orient bounding boxes, classes and confidences. This model is a pre-trained target dataset (DOTA) and was frozen during RL training. This ensures the RL model training based on a stable environment, reducing the complexity of training.

The Reinforcement Learning Model

- The agent is the “brain” of MARS. It is implemented by a Q-Table model.
- At each step, the agent receives the state and selects an action.
- The action was taken by the environment, leading to a new state and a reward.
- The goal of the agent is to learn a policy that maximizes the reward, effectively learning to find the most objects with the least actions.

1.4.2 Operational Workflow

The detecting process for a single image is as follows.

Initialization: The source image is fed to the VM to obtain an initial detection (scale level = 1). The agent’s initial window is set to the center of the image.

Active Analysis Loop: For each step t :

1. The VM processes the image patch at current window and returns detections.
2. The agent’s state is updated with these new results, and the objects discovered are inspected and compiled into a list.
3. The agent selects an action (e.g., moving left, moving right, moving up, moving down, zooming in or zooming out) based on the given action spaces provided by the environment.
4. The environment moves the window, accordingly, extracts the new patch, calculates the reward, and updates the available action spaces.
5. The experience state data class is stored for training.

Termination: The agent continues this process until it finds no action space or reaches a predefined maximum limit.

Result Aggregation: All detection results from each step are combined using Intersection of the Union area (IoU) to remove duplicates and produce the final output for the image.

1.4.3 Methodology for Addressing Aims and Objectives

The solution will be designed, implemented and validated through the following methodology:

Design and Implementation (Obj. 1 and 2): The framework will be built in Python with the custom visual environment. A pre-trained YOLOv11 orient bounding box model will be treated as the frozen Visual Model and the baseline for comparison. The RL agent will be implemented using a Q-table model.

Evaluation and Comparison (Obj. 3 and 4): Both models will be evaluated by the DOTA V1.0 validation dataset. Accuracy will be measured by mAP@0.5. Computational resources will be compared by average predicting time per image, and peak GPU memory usage. More importantly, the prediction time for MARS will be the total time for the entire process.

Policy Analysis (Obj. 5): The agent's policy will be visualized by a visual webpage. This will provide qualitative insights into its decision-making strategy, showing if it learns to ignore areas and focus on new, unexplored areas.

1.4.4 Expected Outcome and Validation

The hypothesis is that MARS will achieve a higher **mAP** than the static baseline, particularly on small and occluded object classes, by actively seeking out and confirming ambiguous targets. Furthermore, it is expected that this performance gain will be achieved with a linear increase in computational costs relative to processing the whole image required for those small objects. The agent's learned policy will be the key factor to this efficiency, validating the core premise of the project.

1.5 Summary of Contributions and Achievements

This project makes several main contributions to the field of remote sensing image analysis and efficient AI model design. The key achievements and innovations are summarized as follows:

Theoretical and Methodological Contribution: The Proposal of the MARS Framework

- **Novel Paradigm:** This project introduces MARS, a novel multi-model agent framework that reformulates remote sensing image analysis from a static, one-pass process into a dynamic, adaptive, and iterative search paradigm. This paradigm change is inspired by the human visual system's saccadic movements and active perception.
- **Integrated Architecture:** We designed and implemented a closed-loop integration of a Reinforcement Learning (RL) agent with a pre-trained, frozen visual model (VM). This design allows the agent to learn an efficient policy for navigating the image space without the need for end-to-end training of the visual backbone, significantly reducing training complexity and computational resources.

Empirical Contribution: Demonstrated Superior Performance on Challenging Tasks

- **Enhanced Accuracy:** Through extensive evaluation of the benchmark DOTA dataset, the MARS framework achieved a higher mean Average Precision (mAP) compared to the baseline model (YOLOv11). This improvement was very obvious for small, Partially Obscured, and shadowed objects (e.g., small vehicles, ships), directly addressing Research Question 1 and validating our primary hypothesis.

- **Qualitative Insights:** We provided a quantitative analysis of the agent's policy, visually demonstrating that it learns intelligent, human-like strategies. For instance, the agent learns to skip detected regions and dedicates its computational budget to zoom in and focuses on the new, unidentified areas. This achievement fulfills Objective 5.

-

Practical Contribution: The trade-off between performance and computing resources

- **Resource-Friendly AI:** The framework shows that it is possible to enhance model visual capability without the need for complex scaled-up models. By avoiding redundant computations on image patches, MARS presents a viable path toward deploying more vision AI on resource-restricted platforms, a significant step for practical IoT integration. This successfully addresses Research Question 2.

Contribution to the Community

- **Open-Source Implementation:** To facilitate reproducibility and further research in this direction, we will release the full source code of the MARS framework, including the custom RL environment, agent implementations, and evaluation scripts, to the public.
- **Benchmarking:** This study provides a new baseline and a methodological framework for future work on active and efficient perception in remote sensing and other computer vision domains.

In conclusion, this project not only achieves its aims and objectives but also makes substantive contributions by designing and valuating a new adaptive model architecture, showing its benefits for accuracy and efficiency, and offering valuable insights into the policies learned of AI agents for visual tasks.

This report maintains a concise and logical structure with seven chapters. Chapter 2, Literature Review, provides comprehensive research of the existing relevant AI models. Chapter 3, Methodology and Design, details the architectural design and implementation of the MARS framework. Chapter 4, Results and Analysis, this chapter presents the empirical findings of the study. Chapter 5, Discussion and analysis, interprets the results presented in Chapter 4. It discusses the significance of the findings, explaining how the MARS framework achieves its performance gains and efficiency. Chapter 6, Conclusion and Future Work, summarizes the key achievements of the project and suggests promising directions for future research to build upon the foundations laid by this work.

Throughout this report, chapters and sections are referenced explicitly (e.g., Chapter 3, Section 3.2). Figures and tables are also named within their chapters (e.g., Figure 4.1, Table 5.3).

1.6 Organization of the Report

This report maintains a concise and logical structure with seven chapters. Chapter 2, Literature Review, provides comprehensive research of the existing relevant AI models. Chapter 3, Methodology and Design, details the architectural design and implementation of the MARS framework. Chapter 4, Results and Analysis, this chapter presents the empirical findings of the study. Chapter 5, Discussion and analysis, interprets the results presented in Chapter 4. It discusses the significance of the findings, explaining how the MARS framework achieves its performance gains and efficiency. Chapter 6, Conclusion and Future Work, summarizes the key achievements of the project and suggests promising directions for future research to

build upon the foundations laid by this work. Throughout this report, chapters and sections are referenced explicitly (e.g., Chapter 3, Section 1.4.1). Figures and tables are also named within their chapters (e.g., Figure 3.6, Table 4.2).

Chapter 2

Literature Review

2.1 Computer Vision Models in Remote Sensing

The application of Computer Vision to Remote Sensing (RS) has revolutionized Earth observation, including automated scene classification, object detection, and image segmentation. The field has evolved from traditional feature extraction methods to deep learning models, with Convolutional Neural Networks (CNNs) now representing the state-of-the-art.

An important example is the Ultralytics YOLO model series issued by [Khanam and Husain \(2024\)](#), which provide an excellent balance of speed and accuracy for oriented object detection, a critical task for identifying multiple objects in aerial images where alignment is arbitrary. For segmentation tasks, the emergence of foundation models like the Segment Anything Model (SAM) ([Kirillov et al. \(2023\)](#)), has inspired domain-specific adaptations such as Remote Sensing segmentation with Segment Anything Model (SAMRS) ([Wang et al. \(2023\)](#)) and RingMo-SAM ([Yan et al. \(2023\)](#)). These models demonstrate significant potential for segmenting arbitrary objects in RS images, thereby aiding in detailed interpretation.

However, [Janga et al. \(2023\)](#)'s research has shown that these single-pass, static models still struggle to overcome several challenges. These challenges include the accurate detection of small, occluded, and shadowed objects, high computational costs associated with processing very high-resolution images at a fixed scale, and a lack of adaptability to the vast heterogeneity present in RS data (e.g., varying altitudes, sensors, and lighting conditions). This underscores a clear gap in the literature: the need for more adaptive and efficient frameworks that can dynamically allocate computational resources.

2.2 Agent-based Systems

A popular trend is the rise of agent frameworks that use Large Language Models (LLMs) as central planners to orchestrate specialized tools, such as RS-Agent. [Xu et al. \(2023\)](#) employed an LLM as a "central controller" that interprets complex user queries, programmatically invoke domain-specific models (e.g., a YOLO detector), and synthesizes answers through multi-step reasoning.

While RS-Agent focuses on high-level task planning and tool invocation, its core principle is highly relevant: breaking down a complex problem into a sequence of simpler, managed steps. The proposed MARS framework shares this foundation but differs fundamentally in its implementation. Instead of using an LLM for linguistic reasoning, MARS employs a specialized RL agent for visual reasoning. Its objective is not to answer a textual query but to exhaustively and accurately explore an image through a learned policy of visual actions.

2.3 Remote Sensing Dataset

Dataset of Object deTecton in Aerial images (DOTA), provided by [Ding et al. \(2021\)](#), is a comprehensive baseline for object detection in aerial images. It contains 11,268 aerial images, including 1,793,658 instances of 18 categories such as ship, bridge and vehicle, etc. This project uses DOTA to train and improve the performance of vision models.

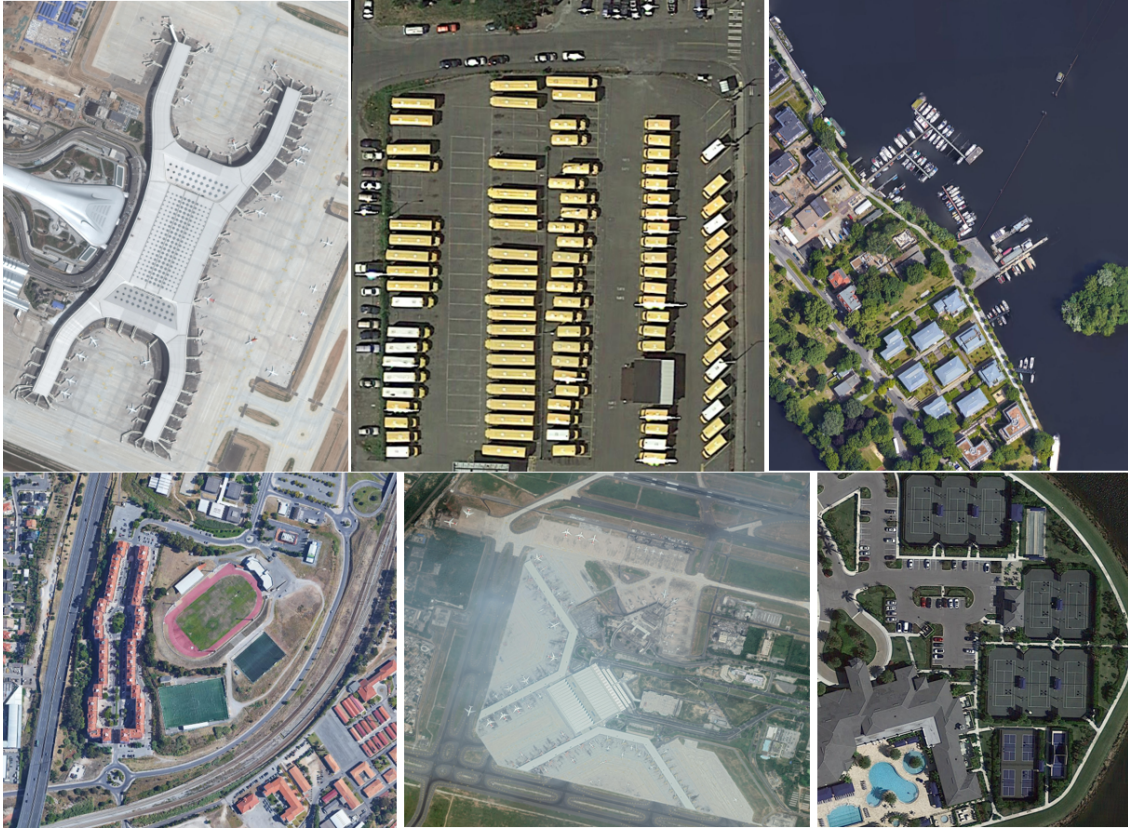


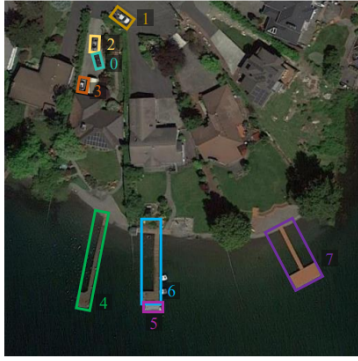
Figure 2.1: Sample Images for DOTA dataset.

[Li et al. \(n.d.\)](#) proposes a large-scale, open source benchmark for object Detection in Optical Remote sensing images, named DIOR. This dataset contains 23463 images of 20 object categories and each category includes about 1200 images.



Figure 2.2: Sample images for DIOR dataset.

Based on image datasets DOTA and DIOR, Li et al. (2024) introduces a new benchmark dataset, named Versatile Vision-Language Benchmark Dataset for Remote Sensing Image Understanding, with general purpose, vision-language question-answer pairs for remote sensing images.



Object Referring

Object ID=1: The small vehicle that is the farthest to the top.

Object ID=4: The harbor located on the left side of the scene with multiple docks extending into the water.

Object ID=7: The harbor situated on the right side of the image with a large dock area.

Visual Question Answer

Question: How many harbors are visible? Answer: 3

Question: What is the object located furthest to the top? Answer: Small vehicle

Question: Are the visible vehicles near water? Answer: No

Detailed Captioning

The high-resolution aerial image from GoogleEarth shows a waterfront scene with residential areas and harbor facilities. Three distinct harbors can be seen, one located on the left side and another on the right side of the image. Between them, there are homes with different colored rooftops, green lawns, and driveways. A ship is docked in the central part of the bottom edge, and the water body exhibits gentle ripples. Various small vehicles are scattered throughout the residential area, parked near the houses.

Figure 2.3: Sample image and question pairs for VLBenchmark.

2.4 Critique of Existing Work and the Research Gap

The existing literature shows that Models like YOLO and SAM-derived systems are powerful but inherently static. These methods only process images once and use a fixed resolution; therefore, they are only suitable for objects within a specific size range. For tasks involving a wider range of object sizes, significant adjustments to the sampling strategy and retraining are necessary. For AI Agents, Frameworks like RS-Agent introduce valuable adaptability and sequential reasoning but operate at a high level of abstraction, orchestrating entire models rather than optimizing the fundamental visual perception process itself. Furthermore, language

models require a significant amount of computing resources, which makes them unsuitable for integration into small services such as IoT.

2.5 Summary

In conclusion, significant advancements in visual models can be directly applied to remote sensing images, such as YOLO for object detection and SAMRS for segmentation, which represent the state-of-the-art. However, these static models are limited by their fixed processing scale, struggle with detecting small or obscured objects. To address this problem, an ideal solution would combine the low-level, adaptive visual reasoning of RL with the sequential planning of an agent-based system to achieve a detail-extracted and accurate interpretation of remote sensing images.

Chapter 3

Methodology

In terms of methodology, we will introduce it from the aspects of system structure, document architecture and training and validation datasets.

3.1 System Structure

To obtain a flexible, adaptive and low-resource visual system, this project proposes the Multi-model Agent for Remote Sensing (MARS) framework. MARS is an adaptive reinforcement learning driven system integrated with a stable vision module, designed to mimic the human visual search process to achieve high-precision fine-grained analysis capabilities. Instead of processing an entire image at a fixed, high resolution, a computationally expensive approach, MARS employs an intelligent agent that learns to strategically search on regions of interest, so that distributing computing resources property and gaining reasonable accuracy.

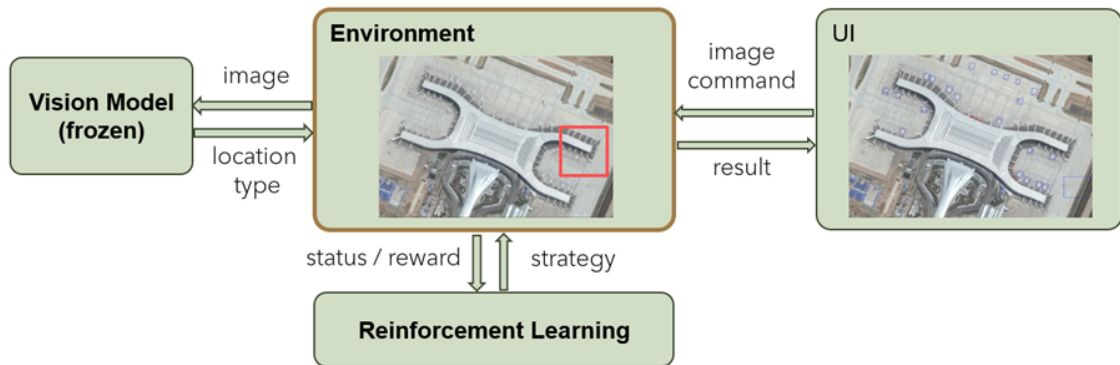


Figure 3.1: The system structure of our MARS framework.

A general workflow of MARS Agent is:

1. An image is uploaded to the agent from the UI service webpage.
2. The agent stores the original image in the environment and calls the visual model for the first recognition, obtaining the state, detection results, a reward and action spaces.
3. Based on the current state and action spaces, the reinforcement learning model generates an action instruction according to the policy.
4. Based on the instruction, the agent will update the state and the visual window to gain a new image patch.

5. Then the image patch is processed by the vision model to identify more new objects and gain action spaces.
6. The reward provided by the Environment is used to update the Reinforcement Learning policy.
7. The agent decides whether to move on to the next recognition cycle by the action spaces.
8. Finally, the agent stops exploring when it reaches a predetermined step or runs out of action space and displays the picture.

3.1.1 Environment: Zoomable Visual Exploration

To investigate and enhance an agent's visual perception and active exploration capabilities, we developed a custom RL environment named the Zoomable Visual Exploration (ZooVE) environment. This environment is implemented as a customizable module adhering to the popular OpenAI Gym API, ensuring compatibility with standard RL algorithms and frameworks.

The Core Components and State Representation of ZooVE environment:

- **Source Image:** A large, high-resolution RGB image which is the entire, static and source image. Source image serves as the ground-truth world the agent must explore.
- **Current Zoom Level:** A set of discrete scaling factors (1, 2, 3 and 4) defining the magnification of the agent's view. A higher zoom level reveals finer details within a narrower field of view, while a lower zoom level provides broader contextual information. For example, 1 represents a zoom ratio of 1, 2 represents a zoom ratio of 0.5, 3 represents a zoom ratio of 0.25, and 4 represents a zoom ratio of 0.125.
- **Window:** A rectangular sub-region, defined by its centroid coordinates (cx, cy) and dimensions (w, h), which is determined by the current zoom level. This represents the agent's current observable window.
- **New Discovered Objects:** A dynamic list of objects [class id, confidence, bounding box] representing new found objects the agent has confidently identified this step. This is used to calculate the reward for the current step.
- **Discovered Objects:** A dynamic list of objects [class id, confidence, bounding box] representing all objects the agent has confidently identified. This acts as a memory module for the agent's achievements. Each new object found will be appended into this list, except that the interference value Uol greater than 0.2 will be merged.
- **Exploration Spaces:** A two-dimensional binary matrix for each zoom level, spatially aligned with the source image, that tracks the explored regions. A variable in the matrix is labeled -1 if no object was detected in the previous step and the variable is within the agent's window; it is labeled 0 if it is not explored yet; otherwise, it is labeled with the confidence value which is greater than 0. This provides a key state variable that encourages efficient exploration and reduces repeated visits to regions.
- **Action Spaces:** The agent can take a customized number of actions to change its field of view. The action space is discrete and includes moving left, moving right, moving up, moving down, zoom in and zoom out. The action spaces provided by the environment build on prior experience to reduce the explosiveness of the exploration space and improve

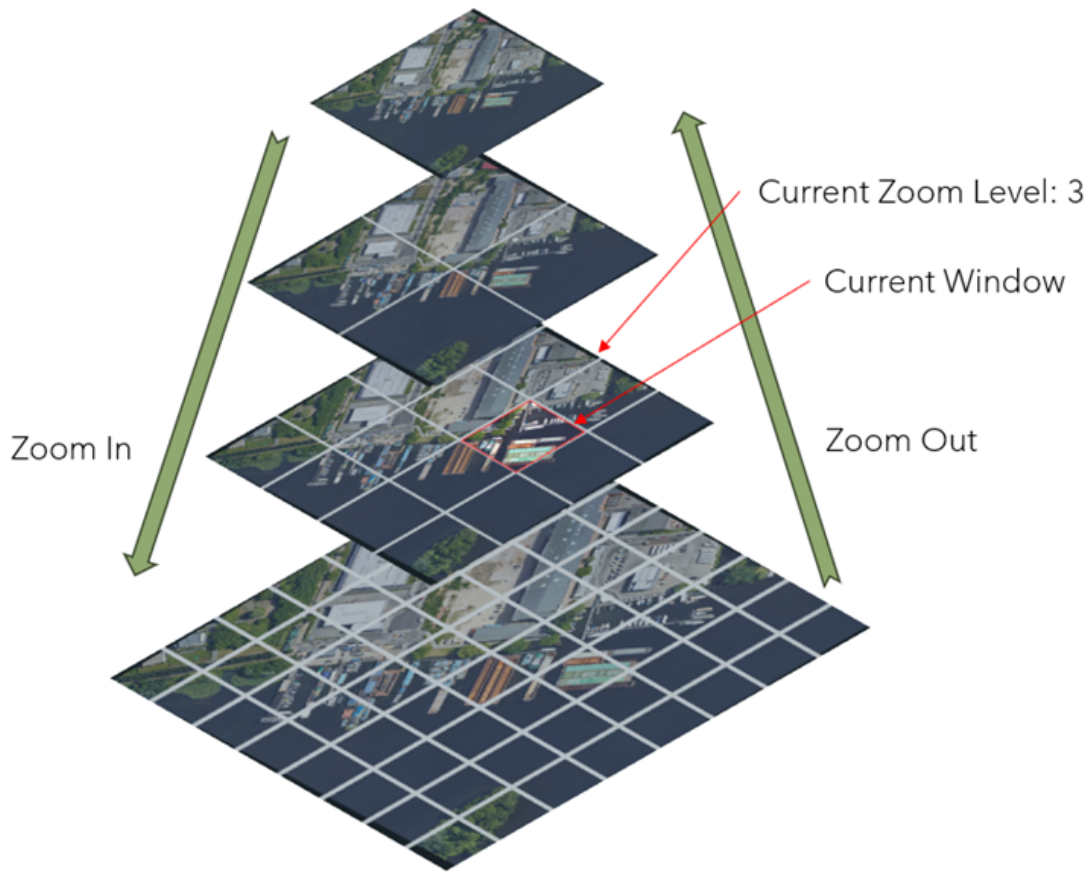


Figure 3.2: The Exploration Space of MARS Environment. The window starts from the top layer (zoom level = 1) and gradually moves to the next step based on the prediction strategy. As the zoom level increases, the area of the corresponding window becomes smaller, and the pixels within the window are magnified, and the corresponding details are enhanced, achieving better recognition results.

exploration efficiency. Constraints include that the window must be within the image bounds and that the viewport cannot repeatedly explore the same location at the same zoom level.

- **Reward Function:** The reward function is essentially for guiding the RL model's learning. It is designed to balance task performance improvement and computational cost. A base positive reward is given for each new object detection action (+1). A smaller positive reward is given for classification confidence. If not found, a small reward (+0.1) for a zoom-in action to encourage detailed searching. A small negative penalty (-0.2) is applied for every action taken without a new object detection.

3.1.2 Reinforcement Learning

The agent serves as the central decision-making module, or the "brain" of the MARS system. It is modeled using a Q-Table. At each timestep, the agent receives a state representation from the environment and uses it to select an action. This action is subsequently executed by the environment, resulting in a transition to a new state and the receipt of a reward signal. The agent's objective is to learn an optimal policy that maximizes its cumulative reward, which corresponds to efficiently discovering the maximum number of objects while minimizing

the number of actions taken.

The Q-table is in key-value format. The key consists of a tuple, including the scale level, indexes in Exploration Matrices. The value is the sum of the bonus from movement.

3.1.3 Vision Model

The VM plays as the “eye” of the agent. Its responsibility is to recognize the objects from the image patch provided by the agent and return the result list, including orient bounding boxes, classes and confidences. This project uses the YOLO v11 ([Khanam and Hussain \(2024\)](#)) orient bounding box model. This model is a pre-trained target dataset (DOTA) and was frozen during RL training. This model was chosen because it combines multi-object detection with a compact size and good performance. It is suitable for running on small GPUs or even in environments without a GPU. This ensures the RL model training based on a stable environment, reducing the complexity of training.

3.1.4 User interface

This project uses the popular AI application framework Gradio to build a web interface for user interaction. It includes a result display area on the left, a window and search map in the upper right corner, a control panel in the middle right, and a message display module in the lower right corner.

A simple workflow involves:

1. The user visits our web service and upload the target image.
2. The image will be rendered on the left panel.
3. The user then configures the corresponding parameters and clicks the detect button to trigger the recognition process.
4. During the recognition process, the backend agent returns the recognition results, the current window, and the recognition map to the frontend for display in real time.
5. The frontend continues to update and display the backend information.
6. The whole recognition process completes.
7. Finally, the web UI shows the detected objects by oriented bounding boxes and a textual message on the message area.

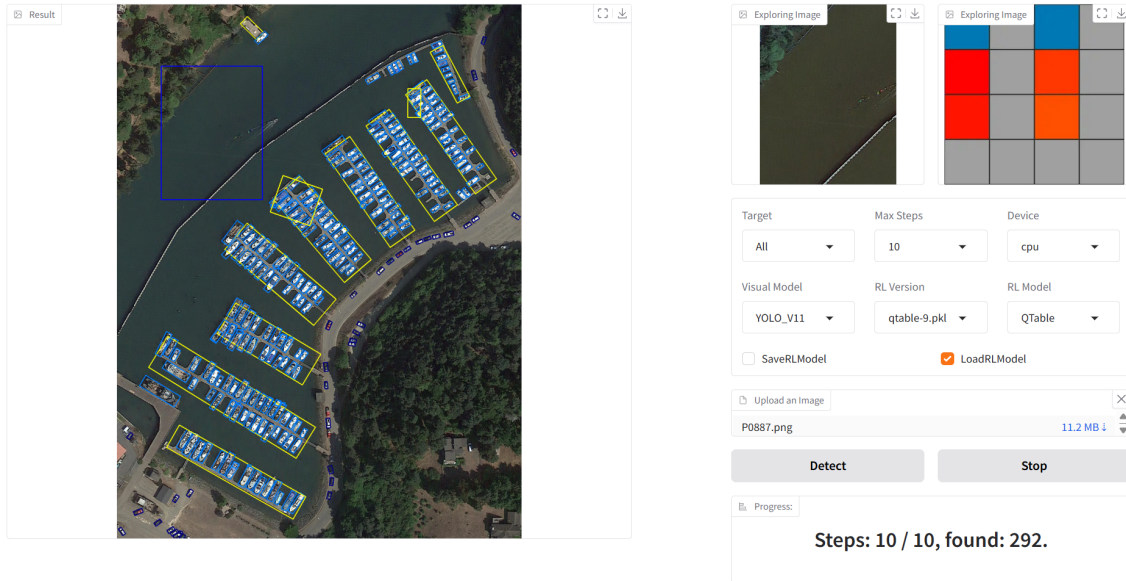


Figure 3.3: The Web Interface of MARS architecture. It includes a result display area on the left, a window and search map in the upper right corner, a control panel in the middle right, and a message display module in the lower right corner.

3.2 Document Architecture

This project is written in Python and the document structure is shown in the figure 3.4. The data folder is used to store data sets, including training, valuation, and testing images and label files. Since there are so many images, we did not put them into a git repository but instead stored them on Google Drive. When using Colab (colab.research.google.com) for training and valuation, we will link to this dataset. The doc folder is used to store introduction documents for this project. RL models will be stored in the models folder. The main Python scripts are placed in the SRC folder, including the visual model script “detector.py”, the reinforcement learning model script “rl-model.py”, the UI application interface “app-v1.py”, and the main entry file of a Jupyter notebook file named “train.ipynb”. Finally, there is also a “readme.md” file to introduce this project.

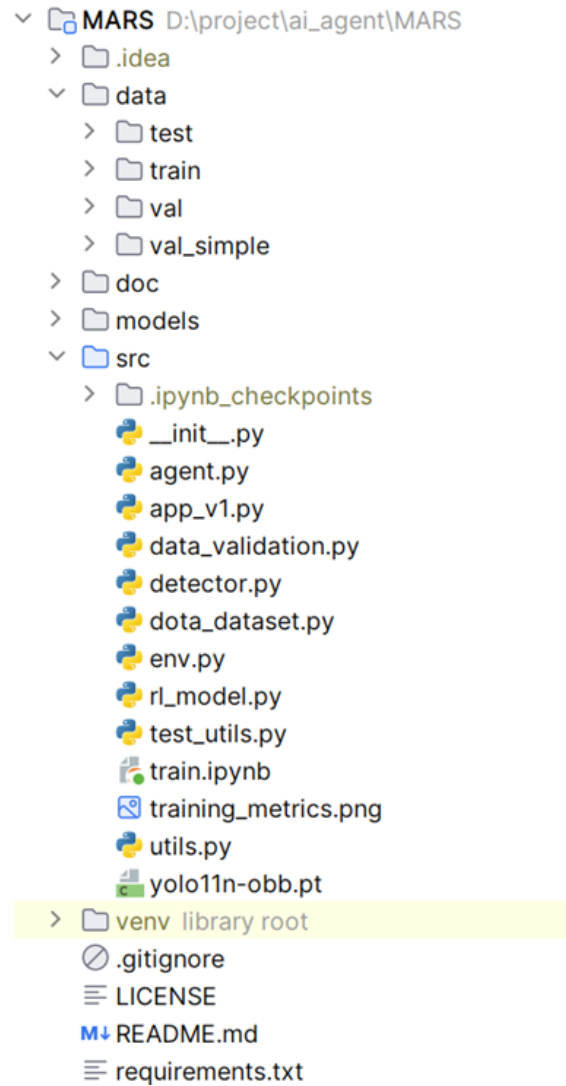


Figure 3.4: Document Architecture of MARS

3.2.1 Introduction of The Main Entry

The main entry file is “train.ipynb”. It includes several sections: Environment Setup: Used to set up the corresponding environment, including installing third-party Python packages, referencing packages, setting dataset paths, and hot-loading Python files. Exploratory Data Analysis: Used to view the type of distribution of training data and object size statistics. Modeling: To manage the DOTA dataset, build the YOLO V11 model, and build and train the MARS Agent. Evaluation: Used to evaluate model performance.

Table of contents	
1	Environment Setting
1.1	Install Required Packages
1.2	Import libraries
1.3	Adapting to the Colab Operating Environment
1.4	Hot Loading .py File
2	Exploratory data analysis
2.1	Object Category Distribution
2.2	Object Bounding Box Sizes Per Category
3	Modelling
3.1	DOTA dataset
3.2	You Only Look Once Model (YOLO V11)
3.2.1	Define Model
3.2.2	Model Evaluation
3.3	MARS Agent
4.2.2	Evaluation
4	Evaluation
4.1.1	Simple datasets
4.1.2	Complex Images
5	Summary

Figure 3.5: Main Entry of MARS

Adaption of Local and Colab Development Environment

To ensure efficient execution of the vision model, we used Colab as the training platform. To facilitate debugging of Python scripts, we also needed compatibility with our local development environment. Therefore, we added code that adapts Colab to our local development environment, see 3.1. This effectively allows for local development, leveraging Colab's GPU computing power and Google Drive's storage capacity.

```

1 def is_colab():
2     try:
3         import google.colab
4         return True
5     except ImportError:
6         return False
7
8
9 if is_colab():
10     print('Running in Google Colab and using Google Drive.')
```

```

11 !git clone "https://github.com/WeijieCui/MARS.git" -b main
12 %cd MARS/src
13 used_drive = bool(input("Do you want to use Google Drive? If so, "
14 "please make sure you have prepared the dataset and models
15 appropriately.\n"
16 "Use Google Drive: Y/N: ").lower() == 'y')
17 if used_drive:
18     try:
19         from google.colab import drive
20         drive.mount('/content/drive')
21         DATASET_ROOT_DIR = '/content/drive/MyDrive/MARS/DOta'
22         MODEL_ROOT_DIR = '/content/drive/MyDrive/MARS/models'
23     except ImportError as e:
24         print(f'failed to import google package: {e.msg}')
25         DATASET_ROOT_DIR = '../data'
26         MODEL_ROOT_DIR = '../models'
27 else:
28     print('Running locally.')
29     DATASET_ROOT_DIR = '../data'
30     MODEL_ROOT_DIR = '../models'
31 else:
32     print('Not running in Colab.')
33     DATASET_ROOT_DIR = '../data'
34     MODEL_ROOT_DIR = '../models'

```

Listing 3.1: Code snippet in Adaptation to Colab

Training RL Q-table Model

This project trained the MARS Q-table model for 50 episodes on Google Colab. For the code, see code snippet 3.2. It used the selected complex training and valuation dataset as the source to train and valuate the model. At each episode, number of steps, rewards and objects detected were collected to quantitatively evaluate model performance. The performance will be printed to the terminal at each episode and saved to a list for subsequent statistics and plotting.

```

1 episodes = 50
2 log_interval = 5
3 MAX_NUM_EXPLORING_STEP = 20
4 rl_qtable_model = rl_model.RLQtableModel(save=True, load=True)
5
6
7 # data for performance figures
8 total_train_rewards = []
9 total_train_vehicles_found = []
10 total_train_avg_episode_steps = []
11
12 total_val_rewards = []
13 total_val_vehicles_found = []
14 total_val_avg_episode_steps = []
15
16 mars_agent = agent.MARSAgent(rl_model=rl_qtable_model, device=device)
17
18 for episode in range(episodes):
19     episode_train_vehicles_found = 0
20     episode_train_reward = 0
21     episode_train_steps = 0
22     count = 0

```



```

23     start = time.time()
24     for image, target in train_dataset:
25         if image is None:
26             print('empty image at count: {}'.format(count))
27             continue
28         count += 1
29         step, image_reward, obbs = mars_agent.visual_search(image, target,
30             max_step=MAX_NUM_EXPLORING_STEP, learn=True)
31         # record this round
32         episode_train_steps += step
33         episode_train_reward += image_reward
34         episode_train_vehicles_found += len(obbs)
35         print(f'\rprocessed {count + 1} images.', end='', flush=True)
36     total_train_rewards.append(episode_train_reward / count)
37     total_train_vehicles_found.append(episode_train_vehicles_found / count)
38     total_train_avg_episode_steps.append(episode_train_steps / count)
39
40     # Valuate
41     episode_val_vehicles_found = 0
42     episode_val_reward = 0
43     episode_val_steps = 0
44     start = time.time()
45     count = 0
46     for image, target in val_dataset:
47         if image is None:
48             print('empty val image at count: {}'.format(count))
49             continue
50         count += 1
51         step, image_reward, obbs = mars_agent.visual_search(image, target,
52             max_step=MAX_NUM_EXPLORING_STEP, learn=True)
53         # record this round
54         episode_val_steps += step
55         episode_val_reward += image_reward
56         episode_val_vehicles_found += len(obbs)
57         print(f'\rprocessed {count + 1} images.', end='', flush=True)
58     total_val_rewards.append(episode_val_reward / count)
59     total_val_vehicles_found.append(episode_val_vehicles_found / count)
60     total_val_avg_episode_steps.append(episode_val_steps / count)
61
62     # Print logs regularly
63     print(f"Episode {episode:4d}/{episodes} | "
64         f"Reward: {episode_train_reward:6.1f} | "
65         f"Vehicles Found: {episode_train_vehicles_found:2d} | "
66         f"Steps: {episode_train_steps}")
67     rl_qtable_model.save('qtable-{}.pkl'.format(episode))

```

Listing 3.2: Code snippet in training RL Q-table model

3.2.2 Introduction of RL model

This project primarily uses Q-table modeling. This model is very simple, using a key-value relationship to manage state and Q-values, see the code snippet at Code List 3.3. When selecting an action, a random motion with a probability of 0.1 is considered. Otherwise, the action with the highest Q-value is selected from the available options. After the MARS agent executes the action, the reward returned by Environment is returned to the RL model, and the RL model updates the Q-value for a specific status based on the reward. In addition, we also support saving and loading models. This facilitates model reusing. In the MARS service,

you can select the model version to use to decide which version to use. If no special version is specified, the "qtable.pkl" version is used by default.

```

1 class RLQtableModel(ReinforcementLearning):
2
3     def __init__(self, eps=0.15, alpha=0.5, gamma=0.9, save=False, load:
4         bool = False, model: str = 'qtable.pkl'):
5         """Init function for RL Q-table model"""
6         # (i,j,scale_bin,action, confidence) -> Q
7         self.model: Dict[Tuple[int, int, int, str, bool], float] = {}
8         # Omitted lines
9
10    def _key(self, i, j, scale_bin, a, avg_conf: float):
11        """Generate a key for Q-table"""
12        return (i, j, scale_bin, a, avg_conf < 0.3)
13
14    def select_action(self, i, j, scale_bin, actions, avg_conf) -> str:
15        """Select an action based on the provided status"""
16        # greedy
17        if random.random() < self.eps:
18            return random.choice(actions)
19        # Select an action with max Q, if not seen, randomly choice one.
20        best_a, best_q = None, -1e9
21        for a in actions:
22            q = self.model.get(self._key(i, j, scale_bin, a, avg_conf),
23                0.0)
24            if q > best_q:
25                best_q, best_a = q, a
26        return best_a or random.choice(actions)
27
28    def update(self, status, a, r, status_new):
29        """Update Q-table based on a reward"""
30        # Omitted lines
31        q_vals = [self.model.get(self._key(i2, j2, scale_bin2, ap,
32            avg_conf2), 0.0) for ap in actions2]
33        max_next = max(q_vals) if q_vals else 0
34        # New QValue = QValue + alpha * (reward + gamma * max_next - Q)
35        new_q = q + self.alpha * (r + self.gamma * max_next - q)
36        self.model[key] = new_q
37
38    def save(self, filename: str = 'qtable.pkl'):
39        """
40        save model
41        :param filename: filename
42        """
43        full_path = os.path.join(MODEL_DIR, filename)
44        if filename.endswith('.pkl'):
45            with open(full_path, 'wb') as f:
46                pickle.dump({
47                    'model': self.model,
48                    'eps': self.eps,
49                    'alpha': self.alpha,
50                    'gamma': self.gamma,
51                }, f)
52        elif filename.endswith('.json'):
53            # Save Q table in JSON format
54            qtable_serializable = {
55                f"{k[0]},{k[1]},{k[2]},{k[3]}": v for k, v in self.model.
56                items()
57            }
58            data = {

```

```

55         'Q': qtable_serializable,
56         'eps': self.eps,
57         'alpha': self.alpha,
58         'gamma': self.gamma,
59     }
60     with open(full_path, 'w') as f:
61         json.dump(data, f, indent=2)
62     else:
63         raise ValueError("format must be 'pickle' or 'json'")
64     print(f"Q table was saved to {full_path}.")
65
66     def load(self, filename: str = 'qtable.pkl'):
67         """
68         load a model from file
69         :param filename: filename
70         :return:
71         """
72         full_path = os.path.join(MODEL_DIR, filename)
73         try:
74             if filename.endswith('.pkl'):
75                 with open(full_path, 'rb') as f:
76                     data = pickle.load(f)
77                     self.model = data['model']
78                     self.eps = data.get('eps', self.eps)
79                     self.alpha = data.get('alpha', self.alpha)
80                     self.gamma = data.get('gamma', self.gamma)
81             elif filename.endswith('.json'):
82                 with open(full_path, 'r') as f:
83                     data = json.load(f)
84                     # Convert json data to an object
85                     self.model = {}
86                     for key_str, value in data['model'].items():
87                         parts = key_str.split(',')
88                         key_tuple = (int(parts[0]), int(parts[1]), int(
89                             parts[2]), parts[3])
90                         self.model[key_tuple] = value
91                     self.eps = data.get('eps', self.eps)
92                     self.alpha = data.get('alpha', self.alpha)
93                     self.gamma = data.get('gamma', self.gamma)
94                 print(f"Load a model from {full_path}.")
95         except FileNotFoundError:
96             print(f"Load a model failed: {full_path} not exist.")
97         except Exception as e:
98             print(f"Load a model failed, message: {e}")

```

Listing 3.3: Code snippet in RL Q-table model

3.2.3 Introduction of MARS Environment

The environment was used to manage the current original image, the current window, exploration footprints and discovered items, and calculate rewards, see the code at Code Listing 3.4. It contained "`__init__()`" and "`reset`" functions to initialize and reset the state. The "`step()`" function was used to process an action and calculate the reward based on this action. During the process, status including the window, footprints and discovered object list were updated, as well as the available action spaces for the next step, calculated by "`refresh_actions()`" function.

```

1 class SearchEnv:

```

```

2     def __init__(self):
3         # Initialize status
4         pass
5
6     def reset(self):
7         """Reset environment status"""
8         # 1 Initialize visual window and zoom level
9         self.found_objects = []
10        self.new_found_objects = []
11        # 2 Initial status and clear detections
12        self.set_image(self.img)
13        self.actions = ['zoom_in']
14        i, j = self._cell_of(self.cx, self.cy)
15        x1, y1, x2, y2 = self._fit_window()
16        crop = self.img[y1:y2, x1:x2]
17        reward = self.first_view()
18        return (i, j, self.scale_idx, self.actions, 0), reward, self.
found_objects, self.new_found_objects, crop
19
20
21    def step(self, action: str):
22        """Step an action and update status"""
23        assert action in self.actions
24        # 1 Update visual window and zoom level
25        step_px = max(20, self.scale_levels[self.scale_idx] * self.W)
26        step_py = max(20, self.scale_levels[self.scale_idx] * self.H)
27        if action == "up":
28            self.cy = self.cy - step_py
29            if self.cy <= 0:
30                self.cy = 0
31        elif action == "down":
32            self.cy = self.cy + step_py
33            if self.cy >= self.H - 1:
34                self.cy = self.H - 1
35        elif action == "left":
36            self.cx = self.cx - step_px
37            if self.cx <= 0:
38                self.cx = 0
39        elif action == "right":
40            self.cx = self.cx + step_px
41            if self.cx >= self.W - 1:
42                self.cx = self.W - 1
43        elif action == "zoom_out":
44            self.scale_idx += 1
45            if self.scale_idx > len(self.scale_levels) - 1:
46                self.scale_idx = len(self.scale_levels) - 1
47        elif action == "zoom_in":
48            self.scale_idx -= 1
49            if self.scale_idx < 0:
50                self.scale_idx = 0
51        self.refresh_actions()
52        # New status
53        i, j = self._cell_of(self.cx, self.cy)
54        # 1) Detect in current window
55        x1, y1, x2, y2 = self._fit_window()
56        # 2 Call a vision model for detection
57        crop = self.img[y1:y2, x1:x2]
58        obbs, _ = self.detector.infer_obb(crop, self.target)
59        fix_obbs(obbs, x1, y1)
60        self.new_found_objects = merge_bounding_box(self.found_objects,
obbs)

```

```

61     self.found_objects.extend(self.new_found_objects)
62     scores = [obb['score'] for obb in self.new_found_objects]
63     best_conf = max(scores) if scores else -1
64     # Update
65     if best_conf > 0:
66         self.multi_grids[self.scale_idx][i, j] = max(self.multi_grids[
self.scale_idx][i, j],
67                                                         best_conf) #
68     Found: update Q score
69     else:
70         if self.multi_grids[self.scale_idx][i, j] == 0 or len(self.
new_found_objects) == 0:
71             self.multi_grids[self.scale_idx][i, j] = -1 # Not found
72         # 3 Calculate reward
73         reward = 0.0
74         if best_conf > 0:
75             reward += 1.0 + best_conf # Found
76             if self.scale_idx < len(self.scale_levels) - 1:
77                 reward += 0.1 # Encourage fine grants
78         else:
79             reward -= 0.2
80             if self.multi_grids[self.scale_idx][i, j] != 0 and self.
multi_grids[self.scale_idx][i, j] != -1:
81                 reward -= 0.1 # In case of overwrite
82             avg_conf = sum(obj['score'] for obj in self.new_found_objects) /
len(self.new_found_objects) if len(
83                 self.new_found_objects) else 0
84             # 4 Return status, detections, reward
85             return (i, j, self.scale_idx, self.actions, avg_conf), reward,
self.found_objects, self.new_found_objects, crop
86
87 def refresh_actions(self):
88     idx_y, idx_x = self._cell_of(self.cx, self.cy)
89     grid_size = self._get_grid_size()
90     grid = self.multi_grids[self.scale_idx]
91     if 'up' in self.actions:
92         if idx_y <= 0 or grid[idx_y - 1, idx_x] != 0 or self.cy <= 0:
93             self.actions.remove('up')
94         elif idx_y > 0 and grid[idx_y - 1, idx_x] == 0 and self.cy > 0:
95             self.actions.append('up')
96     if 'down' in self.actions:
97         if idx_y >= grid_size - 1 or grid[idx_y + 1, idx_x] != 0 or
self.cy >= self.H - 1:
98             self.actions.remove('down')
99         elif idx_y < grid_size - 1 and grid[idx_y + 1, idx_x] == 0 and
self.cy < self.H - 1:
100             self.actions.append('down')
101     if 'left' in self.actions:
102         if idx_x <= 0 or grid[idx_y, idx_x - 1] != 0 or self.cx <= 0:
103             self.actions.remove('left')
104         elif idx_x > 0 and grid[idx_y, idx_x - 1] == 0 and self.cx > 0:
105             self.actions.append('left')
106     if 'right' in self.actions:
107         if (idx_x >= grid_size - 1) or (grid[idx_y, idx_x + 1] != 0)
or (self.cx >= self.W - 1):
108             self.actions.remove('right')
109         elif idx_x < grid_size - 1 and grid[idx_y, idx_x + 1] == 0 and
self.cx < self.W - 1:
110             self.actions.append('right')
111     if 'zoom_out' in self.actions:
112         if self.scale_idx < len(self.multi_grids) - 1:

```

```

112         iu, ju = self._cell_of_zoom_out(self.cx, self.cy)
113         grid_out = self.multi_grids[self.scale_idx + 1]
114         if grid_out[iu, ju] != 0:
115             self.actions.remove('zoom_out')
116         else:
117             self.actions.remove('zoom_out')
118     else:
119         if self.scale_idx < len(self.multi_grids) - 1:
120             iu, ju = self._cell_of_zoom_out(self.cx, self.cy)
121             grid_out = self.multi_grids[self.scale_idx + 1]
122             if grid_out[iu, ju] == 0:
123                 self.actions.append('zoom_out')
124         if 'zoom_in' in self.actions:
125             if self.scale_idx > 0:
126                 iu, ju = self._cell_of_zoom_in(self.cx, self.cy)
127                 grid_in = self.multi_grids[self.scale_idx - 1]
128                 if grid_in[iu, ju] != 0:
129                     self.actions.remove('zoom_in')
130             else:
131                 self.actions.remove('zoom_in')
132         else:
133             if self.scale_idx > 0:
134                 iu, ju = self._cell_of_zoom_in(self.cx, self.cy)
135                 grid_in = self.multi_grids[self.scale_idx - 1]
136                 if grid_in[iu, ju] == 0:
137                     self.actions.append('zoom_in')

```

Listing 3.4: Code Snippet in MARS Environment

3.3 Datasets

To verify the visual capabilities of the model, we selected the mainstream remote sensing image database DOTA (Dataset for Object deTection in Aerial images). DOTA v1.0 is a large-scale benchmark dataset designed for evaluating object detection algorithms in aerial imagery. It is widely recognized in the computer vision and remote sensing communities for its complexity, high resolution, and the diverse range of objects it contains. It contains 2,800+ high-resolution images (800×800 to $20,000 \times 20,000$ pixels) from multiple sources like Google Earth and satellites, with nearly 190,000 annotated object instances across 15 categories (e.g., plane, ship, vehicle, sports fields). This project focuses on particularly dense images, mainly including small cars, big cars and ships.

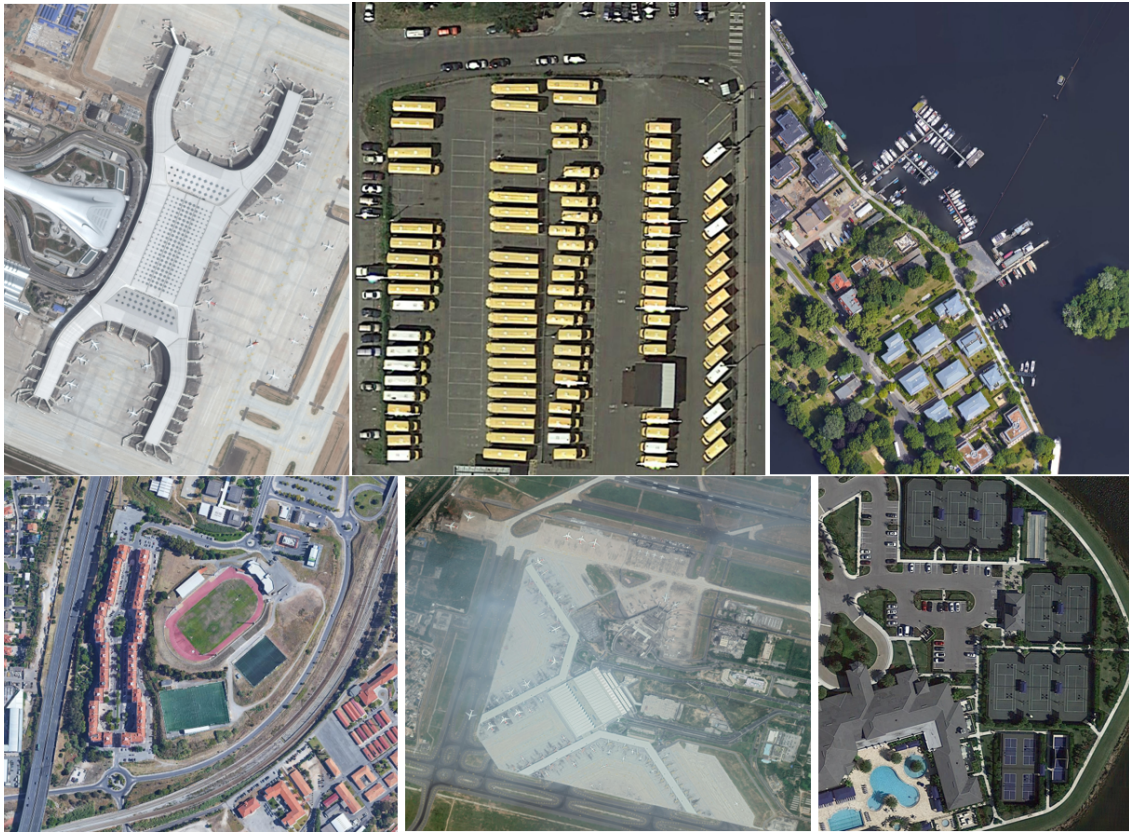


Figure 3.6: Dense urban example, where the main targets are swimming pools and cars. And an example of a dense parking lot, where the main targets are small and large cars.



Figure 3.7: An example of a sports center including football fields, badminton courts, swimming pools, and small cars. And an example of a body of water, including ships, harbors, and small cars on the ground.

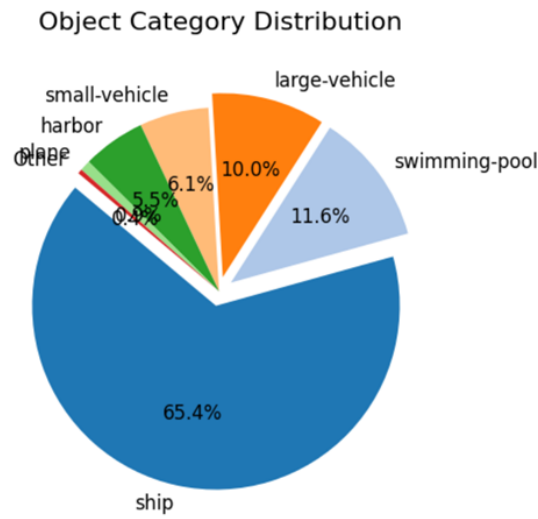


Figure 3.8: Object type distribution diagram. Mainly including ships, swimming pools, large vehicles, small cars, etc.

3.4 Testing and Validation

The standard evaluation protocol for DOTA v1.0 follows the common practices in object detection, using Intersection over Union (IoU) and mean Average Precision (mAP) as the primary metrics.

- Intersection over Union (IoU): IoU measures the overlap between a predicted bounding box and a ground truth bounding box.
- mean Average Precision (mAP): This is the primary metric for ranking and comparing models on DOTA. It is computed by taking the mean of the AP scores across all 15 object classes.

$$s = \frac{1}{15} \sum_{i=1}^{15} AP_i. \quad (3.1)$$

3.5 Summary

This project introduces the MARS framework, an adaptive system designed for efficient, high-precision analysis of large aerial images. Instead of processing an entire high-resolution image—a computationally expensive task—MARS mimics the human visual search process. It uses a RL agent that learns to strategically explore only regions of interest, thereby distributing computing resources wisely.

The system's core is a custom RL environment called Zoomable Visual Exploration (ZooVE). Here, an agent navigates a large source image through a movable and zoomable window. Its action space includes moving up, down, left, right, zooming in, and zooming out. The agent's goal is to maximize its cumulative reward, which is earned primarily by identifying new objects (using a pre-trained YOLO vision model) and is penalized for inefficient, repetitive actions. The environment tracks explored areas using an exploration matrix to prevent redundant searches.

The agent itself, the "brain" of MARS, has been implemented as a Q-Table. It makes decisions based on the current state of ZooVE, which includes the zoom level, window position,

and the history of discovered objects. A web interface built with Gradio allows users to upload images, configure parameters, and view the recognition process and results in real-time. The system was validated using the DOTA v1.0 dataset, a large-scale benchmark for object detection in aerial imagery, demonstrating its capability for fine-grained, low-resource visual analysis.

Chapter 4

Results

4.1 Metrics for Training RL Model

We used the DOTA v1.0 dataset to train and evaluate our RL model. We used frozen Yolo v11 model as our vision model, so we just picked 30 complex images from the training dataset to build the training dataset and picked 8 complex images from the valuation dataset to build the validation dataset. During the training process of the RL model, we always use the frozen visual model YOLO v11. This can maintain a stable performance of the vision model throughout the process and reduce variable factors. This can help to reduce the number of training episodes. We used a T4 GPU on the Google Colab platform to train the model for 50 episodes, which took about 2 hours to run. Please refer to the code for more details at the notebook file named "train.ipynb".

The left side of Figure 4.1 shows the average number of vehicles found by MARS Agent in each episode in the training process. During training, the total number of targets discovered per episode remained relatively stable, around 280 targets discovered in the training dataset and 210 targets discovered in the validation dataset.

The right side of Figure 4.1 shows the average reward per image in each episode in the training process. During training, the model's reward on the training dataset was about 12, and its performance on the validation dataset was in the range of 10 to 13.

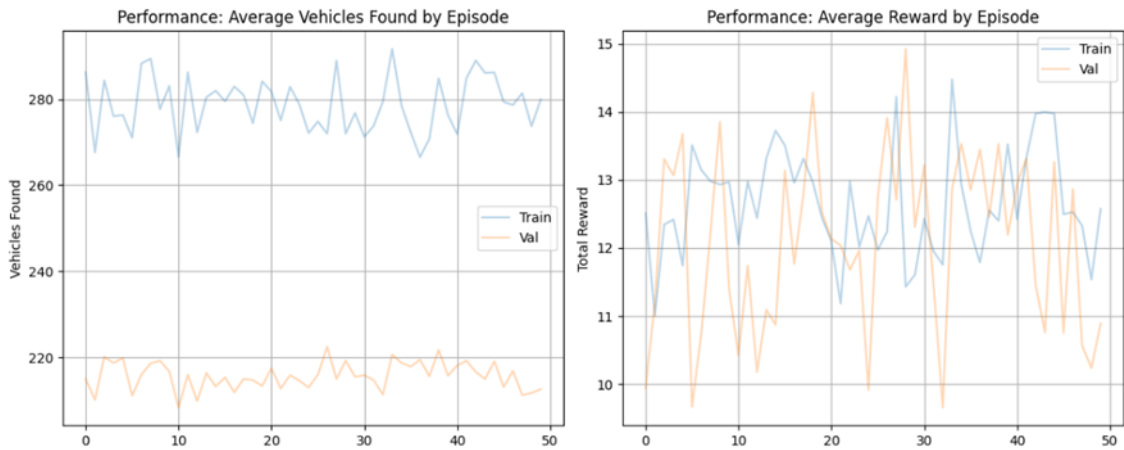


Figure 4.1: Performance of RL Q-table Model-1. Left: Average vehicle found in each episode. Right: Average Reward in each episode.

The left side of Figure 4.2 shows the average number of steps per image for the ex-

ploration process of MARS Agent in each episode in the training process. During training, the average number of steps for the training images was about 12 to 13. For the valuation images, the average number of steps was between 10.5 and 12.5, and there was a downward trend to 10 during the last 5 episodes.

The right side of Figure 4.2 shows the average objects found per step in each episode in the training process. The model discovered about 22.5 objects each step for the training dataset and about 19 objects for the valuation dataset and in the last 5 episodes there was a slight improvement to 21.

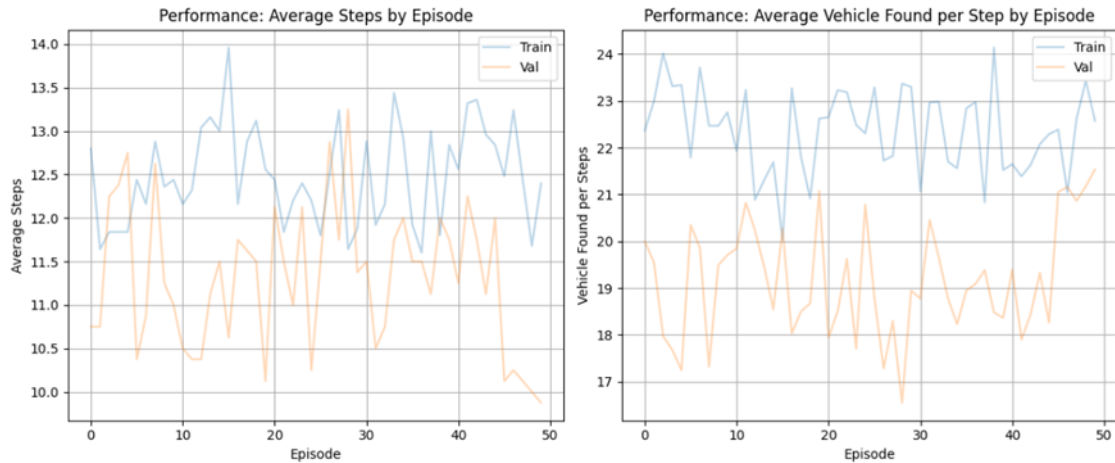


Figure 4.2: Performance of RL Q-table Model-2

In general, the training of this QL model basically achieved the goals set for this project and achieved certain performance improvements within a limited number of steps.

4.2 Benchmark and Metrics for MARS Agent

We employed the selected simple and complex images from DOTA to evaluate our proposed method. To facilitate the evaluation of the results, we used Yolo v11 as a reference. The data was divided into indifferent validation data from the DOTA validation dataset and a specially selected high-density task dataset. The dataset contains 25 validating images. For the simple images, the MARS(Ours) performance improved from an average accuracy of 0.331 to 0.337 comparing to the base model YOLO v11. Specifically, for small-vehicle category, it improves the model accuracy from 0.151 to 0.162, which is about 7.3 percentages improvement.

Table 4.1 shown the mAP@0.5 performance of MARS agent and YOLOv11 in 458 images in DOTA validation dataset (part1). The mAP for all 15 categories of MARS was 0.326, performed 0.045 higher than YOLOv11, which was about 16 percent improvement. For more details, please check A.1 in appendix A.

Table 4.1: mAP@0.5 Performance of MARS Agent and YOLOv11 in Simple Images

	mAP@0.5	Small Vehicle	Large Vehicle	Ship	Others
MARS(ours)	0.326	0.201	0.061	0.048	-
YOLOv11	0.281	0.156	0.066	0.052	-

Table 4.2 shown the mAP@0.5 performance of MARS agent and YOLOv11 in 8 selected

complex images in DOTA validation dataset. The mAP for all 15 categories of MARS was 0.026, performed 0.001 higher than YOLOv11. This value was relatively low because the selected complex dataset is relatively small and many types are not in the image, resulting in an AP data of 0, such as bridge and helicopter. For more details, please check [A.2](#) in appendix A.

Table 4.2: mAP@0.5 of Complex Images

	mAP@0.5	Small Vehicle	Large Vehicle	Ship	Others
MARS(ours)	0.026	0.147	0.054	0.113	-
YOLOv11	0.025	0.140	0.062	0.115	-

4.3 Comparison of Visualization Results

Next, we compare the recognition effects of MARS Agent and YOLOv11 in a visual way. The original image is used as the background, and the identified object bounding boxes are marked on the image. Different objects are distinguished by different colors assigned by the service, for example, small cars are dark blue, ships are light blue and docks are yellow.

Figure 4.3 shown sample results at the lake. In this case, the performance of the two models is roughly the same, with MARS having a slight advantage in identifying the cars in the lower left corner.

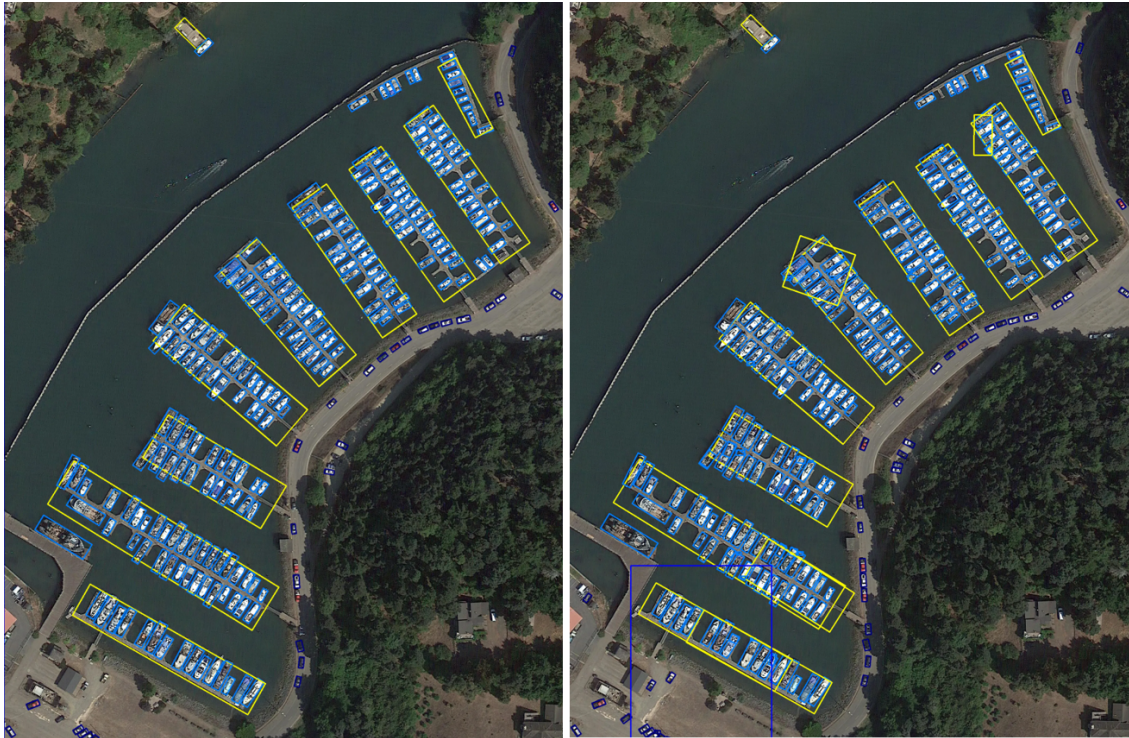


Figure 4.3: Sample Results of YOLOv11 and MARS at the Lake

Figure 4.4 shown sample results in wide waters. For larger vessels, both models performed very well, but MARS was significantly better at identifying small boats near the shore. MARS was also able to identify the large number of cars in the parking lots on the upper left and upper right.



Figure 4.4: Sample Results of YOLOv11 and MARS at Open Waters

Figure 4.5 shown sample results in an airport. The two models perform equally well in recognizing large and small aircraft. However, MARS performs better in recognizing irregular-shaped aircraft and densely packed small aircraft in the upper center of the image.

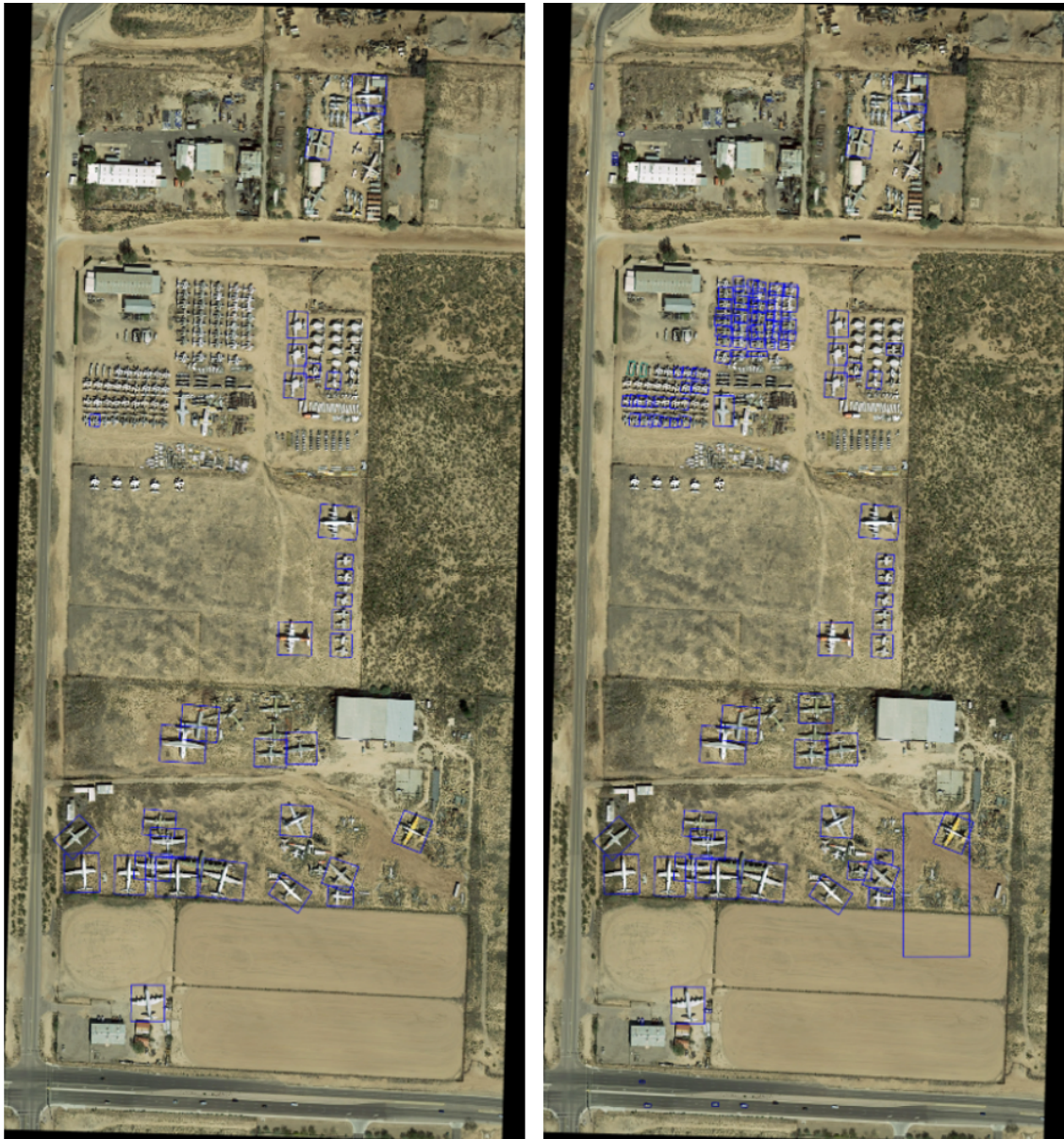


Figure 4.5: Sample Results of YOLOv11 and MARS at an Airport

Figure 4.6 shown sample results in a car park. In this example, both models performed quite well, discovering most of the big cars and small cars in the parking lot.

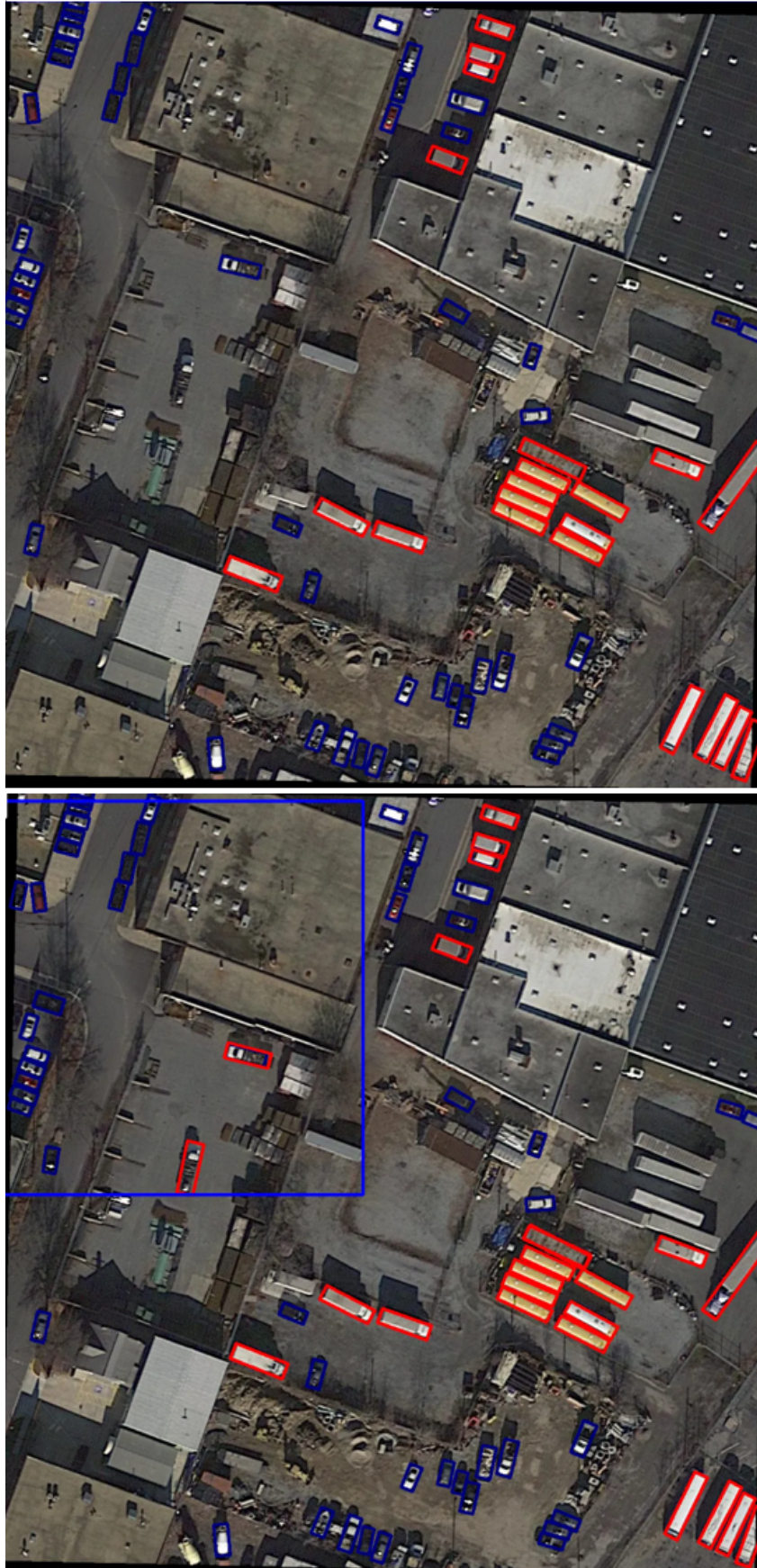


Figure 4.6: Sample Results of YOLOv11 and MARS at an Car Park

4.4 Summary

Overall, during the RL model training process, we found that the RL model's performance met our expectations, being relatively stable and showing a gradual improvement. Most recognitions achieved the expected results within 11 steps. In this overall comparison of recognition performance, we found that both models performed well for appropriately sized, clear objects. However, when it came to recognizing small objects and densely packed groups of objects, MARS was able to uncover more details and achieve better recognition results.

Chapter 5

Discussion and Analysis

Exploring remote sensing imagery is a valuable and challenging task. While mainstream vision models and multimodal language vision models have made significant progress in this area, there's still room for improvement in extracting fine-grained details. The MARS agent's approach of improving detail extraction through multiple explorations has proven effective in experiments. Furthermore, this experiment ran well on Colab, utilizing only a 1.5GB GPU and a 4GB CPU. This small size and low resource requirements make it easier for it to be migrated to other single services.

5.1 Experimental Results

In Chapter 4, we used a complex image dataset to train and validate the MARS agent. We trained the reinforcement learning model for 50 episodes, finding it generally stable and achieving good results within approximately 11 steps. We also compared the performance of MARS with YOLOv11 using a large valuation dataset (458 images in DOTA v1.0 Validation dataset, part 1). The results showed that MARS performed similarly to YOLO in some situations, but significantly outperformed YOLO in specific cases, such as recognizing small objects, partially occluded objects, or densely packed objects.

5.2 Significance of the Findings

This experiment initially demonstrated that MARS Agent has a clear advantage in exploring details in remote sensing images, although there is still a significant speed gap. The idea of breaking down visual tasks into multiple smaller tasks is feasible in practice. It effectively reduces the task requirements for a single model, allowing for improved recognition performance without increasing the model's size.

5.3 Limitations

Currently, only the YOLOv11 visual model is integrated and focuses on classification and positioning tasks. This may reduce the persuasiveness and robustness of the MARS model. In addition, this project uses Q-table as a decision-making model. Compared with deep neural network models, this model is still simple and cannot adapt to more state scenarios, such as the ability to continue exploring regions of interest in a fine-grained manner and the ability to simultaneously identify multiple regions of interest with high concurrency.

5.4 Practical Values

This project demonstrates the feasibility of integrating agents to decompose visual model complexity. This idea can be used to integrate multiple smaller visual models to create a large-scale object recognition system. It can also be used to improve language and vision models by simplifying their need for large amounts of labeled data.

5.5 Summary

The key findings of this project demonstrate that the agent's multi-step exploration strategy is highly effective in extracting fine-grained details from complex remote sensing imagery. The core significance of these findings lies in validating a decomposable approach to vision tasks, although there are some limitations, such as the oversimplification of the Q-table decision model, which cannot adapt to complex states. In summary, the MARS agent demonstrates promising results and proposes a novel approach to detail extraction. Its architecture also points to clear directions for future development, particularly in adopting more advanced decision models.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

In summary, this project proposes an AI agent framework that uses reinforcement learning to improve the recognition capabilities of visual models. This framework uses a reinforcement learning model to provide object search strategies, guiding the visual model to continuously discover objects of interest, thereby improving the number of object recognitions, confidence, and localization performance. This significantly improves the recognition of small-scale, partially occluded, or shadowed objects. This project also provides new directions and insights for future research on remote sensing images: using distributed computing to improve the model's ability to mine detailed information without increasing the instantaneous resource limit. This facilitates breaking down multi-type object recognition tasks into multiple steps, ultimately achieving more refined recognition results.

6.2 Future Work

Guidance on writing future work: This project is just the beginning of remote sensing image recognition, and much work remains to be done. Future work could include adding Scene Recognition models and Panoramic Image Segmentation models, pre-identifying the scene and segmenting the image before distributing the image to the corresponding object recognition model. This has resulted in more specialized and accurate recognition results. For example, if a water area is identified, a model specifically designed to recognize ships, boats, swimmers and beach objects could be used. If a residential area is identified, a model focused on identifying objects such as cars and residential buildings could be used. More rescue scenario models can be integrated into the system to obtain comprehensive software technical support capabilities, such as damaged cars, ship debris or injured people.

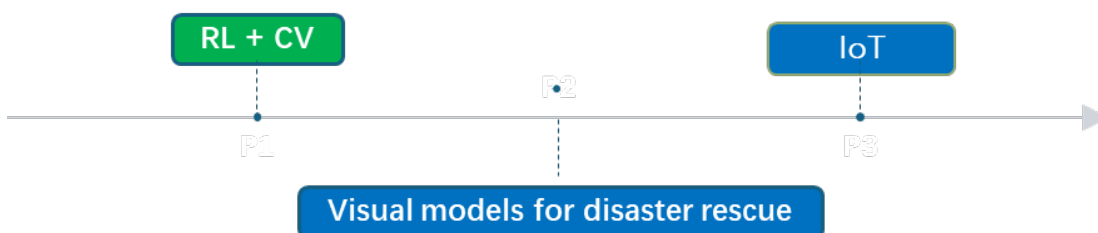


Figure 6.1: Future Work Plan for MARS.

Finally, although many models are integrated, each one is relatively small, so the number of models running simultaneously can be adjusted based on the hardware size. This adjustment can maximize the system's compatibility with small hardware and make it possible to integrate the system into IoT devices.

Chapter 7

Reflection

My main contributions to this project were designing the MARS architecture, setting up the simulation environment, training the reinforcement learning model, and validating the feasibility of using agent AI to improve visual models.

Knowledge and Skills: In order to complete this project, I learned and practiced a lot of AI-related technologies, such as neural network vision models, language vision multimodal models for visual targets, reinforcement learning and AI agents for building and training MARS agent, and gradio for building web pages.

Challenges: In the early stages of the project, I needed to research extensively to understand how to improve remote sensing image recognition. Later on, my main challenge was improving the computation speed and training capabilities of the Q-table. I focused on identifying executable actions within the environment module to reduce repetitive exploration steps.

Different Attempts: If I were to repeat similar work again, I would tend to use a deep neural network instead of a Q-table to get a smarter decision-making brain. This may take more time to build the model and require more computing resources to train the model, but the model can get better generalization ability and be more intelligent.

Divisions: I originally envisioned a 2-3x increase in runtime for the exploration of one image. However, in practice, it can take up to 10 times longer, depending on the steps of exploration. This is primarily because decisions depend on the results of the previous exploration, resulting in a sequential exploration process. Dynamically parallelizing computations may be considered in the future.

References

- Ding, J., Xue, N., Xia, G.-S., Bai, X., Yang, W., Yang, M., Belongie, S., Luo, J., Datcu, M., Pelillo, M. and Zhang, L. (2021), 'Object detection in aerial images: A large-scale benchmark and challenges', *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**, 1–1.
URL: <https://arxiv.org/pdf/2102.12219v1.pdf>
- Janga, B., Asamani, G. P., Sun, Z. and Cristea, N. (2023), 'A review of practical ai for remote sensing in earth sciences', *Remote Sensing* **15**, 4112.
URL: <https://www.mdpi.com/2072-4292/15/16/4112>
- Khanam, R. and Hussain, M. (2024), 'Yolov11: An overview of the key architectural enhancements'.
URL: <https://arxiv.org/pdf/2410.17725>
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., Dollár, P. and Girshick, R. (2023), 'Segment anything', *arXiv:2304.02643 [cs]* .
URL: <https://arxiv.org/abs/2304.02643>
- Li, K., Wan, G., Meng, L. and Han, J. (n.d.), 'Object detection in optical remote sensing images: A survey and a new benchmark'.
URL: <https://arxiv.org/pdf/1909.00133>
- Li, X., Ding, J. and Elhoseiny, M. (2024), 'Vrsbench: A versatile vision-language benchmark dataset for remote sensing image understanding'.
URL: <https://arxiv.org/abs/2406.12384>
- Wang, D., Zhang, J., Du, B., Xu, M., Liu, L., Tao, D. and Zhang, L. (2023), 'Samrs: Scaling-up remote sensing segmentation dataset with segment anything model'.
URL: <https://arxiv.org/abs/2305.02034>
- Xu, W., Yu, Z., Wang, Y. and Wang, J. (2023), 'Rs-agent: Automating remote sensing tasks through intelligent agents'.
URL: <https://arxiv.org/html/2406.07089v1>
- Yan, Z., Li, J., Li, X., Zhou, R., Zhang, W., Feng, Y., Diao, W., Fu, K. and Sun, X. (2023), 'Ringmo-sam: A foundation model for segment anything in multimodal remote-sensing images', *IEEE Transactions on Geoscience and Remote Sensing* **61**, 1–16.

Appendix A

An Appendix Chapter (Optional)

	mAP@0.5	plane	baseball-diamond	bridge	ground-track-field	small-vehicle	large-vehicle	ship	tennis-court	basketball-court	storage-tank	soccer-ball-field	roundabout	harbor	swimming-pool	helicopter
MARS	0.325813	0.800324	0.638150	0.066397	0.227028	0.200726	0.061257	0.048181	0.257172	0.240795	0.652940	0.428755	0.440902	0.137961	0.297601	0.389002
BASE	0.280889	0.716644	0.523498	0.090909	0.214827	0.155790	0.066246	0.052034	0.258717	0.226854	0.532362	0.379674	0.249301	0.113943	0.268899	0.363636

Figure A.1: mAP@0.5 Performance of MARS agent and YOLOv11 base model in valuation dataset

	mAP@0.5	plane	baseball-diamond	bridge	ground-track-field	small-vehicle	large-vehicle	ship	tennis-court	basketball-court	storage-tank	soccer-ball-field	roundabout	harbor	swimming-pool	helicopter
MARS	0.026415	0.0	0.0	0.0	0.0	0.146850	0.053977	0.113952	0.0	0.0	0.0	0.0	0.0	0.081439	0.0	0.0
BASE	0.025039	0.0	0.0	0.0	0.0	0.140404	0.061589	0.114884	0.0	0.0	0.0	0.0	0.0	0.058712	0.0	0.0

Figure A.2: mAP@0.5 Performance of MARS agent and YOLOv11 base model in complex valuation dataset